1) Цель

   Знакомство с принципами организации виртуальной памяти.

2) Задание

   a) В данной работе необходимо реализовать фрагмент диспетчера памяти и часть функционала операционной системы, отвечающего за замещение страниц при возникновении ошибок отсутствия страниц. Для упрощения работы предполагается использование линейной инвертированной таблицы страниц, работу с которой необходимо реализовать в виде программы. Также для простоты предполагается, что в системе имеется один единственный процесс, поэтому идентификатор процесса в инвертированной таблице страниц не хранится. Входные данные представляют собой последовательность операций обращения к памяти, выходные данные - состояние инвертированной таблицы страниц после каждой операции обращения к памяти.

   b) Вычислить номер варианта по списку в журнале и сохранить его в файл TASKID.txt в репозитории.

   c) Написать программу на языке C++ в соответствии со следующей спецификацией.

      (i) Входные данные:

         (1) Аргумент командной строки (число): номер алгоритма замещения страниц, который должна использовать программа. Принимает значения 1 или 2, соответствующие двум алгоритмам замещения страниц, заданным по варианту.

         (2) Перечень инструкций обращения к памяти, считываемый программой из стандартного потока ввода. На каждой строке не более одной инструкции. Инструкция состоит из двух чисел, разделенных пробелом, например: 0 1. Первое число обозначает тип операции доступа к памяти: 0 - чтение и 1 - запись. Второе число является номером виртуальной страницы, к которой происходит обращение.

      (ii) Выходные данные:

         (1) Для каждой операции обращения к памяти, информация о которой поступила на вход программы, на выходе должна быть сгенерирована строка, содержащая содержимое инвертированной таблицы страниц в виде последовательности номеров виртуальных страниц, разделенных пробелом. Если какая-либо из записей в таблице страниц отсутствует (таблица страниц не заполнена до конца), вместо номера виртуальной страницы необходимо вывести символ #.

         (2) Весь код поместить в файле lab4.cpp. Код должен корректно компилироваться командой g++ lab4.cpp -o lab4 -std=c++11. Настоятельно

рекомендуется использовать стандартную библиотеку STL. Полезными могут быть контейнеры list, vector, bitset и др.

d) Если в работе алгоритма замещения страниц используется бит R, то необходимо реализовать эмуляцию прерывания таймера. Для этого через каждые 5 операций обращения к памяти необходимо запускать обработчик данного прерывания. Значения битов R по прерыванию таймера сбрасываются.

e) Для алгоритмов, использующих счетчик (NFU, Aging): если несколько страниц имеют одинаковое значение счетчика, одна из них выбирается случайным образом. При повторной загрузке страницы в память ее счетчик обнуляется. В алгоритме старения счетчик имеет размер 1 байт. В алгоритме NFU счетчик имеет размер не меньше 4 байт.

f) Во всех алгоритмах, использующих датчик случайных чисел (Random, NRU, NFU, Aging, ...), разрешается использовать только функцию int uniform_rnd(int a, int b), объявленную в файле lab4.h. Данная функция генерирует случайное целое число с равномерным распределением из диапазона [a, b]. Использование других функций для работы со случайными числами запрещено!

g) В качестве системного времени в алгоритме рабочего набора следует использовать количество инструкций доступа к памяти, обработанных с момента запуска программы.

h) После успешного прохождения локальных тестов необходимо загрузить код в репозиторий на гитхабе.

i) Сделать выводы об эффективности реализованных алгоритмов замещения страниц. Сравнить количество ошибок отсутствия страниц, генерируемых на тестовых данных при использовании каждого алгоритма.

j) Подготовить отчет о выполнении лабораторной работы и загрузить его под именем report.pdf в репозиторий. В случае использования системы компьютерной верстки LaTeX также загрузить исходный файл report.tex.

3) Алгоритмы замещения страниц

Часы (Clock)

Все страницы сохраняются в циклическом связном списке. При возникновении ошибки отсутствия страниц проверяется бит R у страницы, на который указывает переменная-указатель(стрелка часов), при R=1 бит сбрасывается и стрелка переводится на следующую страницу, так повторяется для каждой страницы, пока не будет найден R=0, который и будет выгружен из памяти.

Наименее востребованная страница (LRU, Least Recently Used)

Все страницы сортируются в порядке обращения к ним. В голове списка страница, к которой обращение было последним . В конце списка страница, к которой

обращение было раньше всех остальных. При каждом обращении к памяти список перестраивается. При возникновении ошибки отсутствия страницы удаляется страница из конца очереди.

4) Результат выполнения работы

На основе проведенных тестов было выявлено, что алгоритм замещения LRU будет эффективнее Clock, так как выгружает страницы по факту обращения и его давности.

5) Исходный код

lab4.cpp

```
#include <iostream>
#include <sstream>
#include <list>
#include <algorithm>
#include "lab4.h"

using namespace std;

// Debug for additional info about process
//#define DEBUG

// Num of page blocks
int PageBlock = 6;
// Read mode
string ReadMode = "0";
// Write mode
string WriteMode = "1";

// Row for table
struct TableRow
{
    string VirPageNumber = "#";
    int R = 0;
};

// Class for two-linked list
class RingLinkedList
{
public:
```

```
RingLinkedList()
{

};

~RingLinkedList()
{
    if (!ListHead)
    {
        return;
    }
    Node* Cur = ListHead->Next;
    while (Cur != ListHead)
    {
        Node* Tmp = Cur;
        Cur = Cur->Next;
        delete Tmp;
    }
    delete ListHead;
};

int GetListSize()
{
    return size;
};

void TableToList(vector<TableRow*>& InPageTableVector)
{
    for (int i=0; i<InPageTableVector.size();i++)
    {
        AddPageToList(InPageTableVector[i], i);
    }
};

int FindPageToChange()
{
    int FindedIndex = -1;
    while (true)
    {
        if (ListHead->VirtualPage->R == 0)
        {
```

```
                FindedIndex = ListHead->IndexInTable;
                #ifdef DEBUG
                    cout << "Finded page to remove " <<
                    ListHead->VirtualPage->VirPageNumber << "at " << FindedIndex
                    << endl;
                #endif
                ListHead = ListHead->Next;

                break;
            }
            else
            {
                ListHead->VirtualPage->R = 0;
                #ifdef DEBUG
                    cout << "Can't remove page " <<
                    ListHead->VirtualPage->VirPageNumber << endl;
                #endif
                ListHead = ListHead->Next;
            }
        }

        return FindedIndex;
    }

private:
    struct Node
    {
        TableRow* VirtualPage;
        int IndexInTable = 0;
        Node* Next = nullptr;
        Node(TableRow* InVirPage, int InPageIndex) : VirtualPage(InVirPage),
        IndexInTable(InPageIndex) {}
    };

    Node* ListHead = nullptr;
    int size = 0;

    void AddPageToList(TableRow* InRow, int InIndex)
    {
        size++;
        Node* NewNode = new Node(InRow, InIndex);
```

```
        if (!ListHead)
        {
            ListHead = NewNode;
            NewNode->Next = NewNode;
            return;
        }

        Node* Cur = ListHead;
        while (Cur->Next != ListHead)
        {
            Cur = Cur->Next;
        }
        Cur->Next = NewNode;
        NewNode->Next = ListHead;
    };
};

// Classes for algorithms
class PageAlgorithm
{
public:
    virtual void LoadPage(string InPage, vector<TableRow*>& PageTableVector,
    int& OutSlotResult)
    {
        #ifdef DEBUG
            cout << "Loading page " << InPage << " to page table" << endl;
        #endif
    };

    virtual void PageHasBeenUsed(TableRow*) {};
};

class ClockAlgorithm: public PageAlgorithm
{
public:
    ClockAlgorithm()
    {
        PageList = new RingLinkedList();
    }
    ~ClockAlgorithm()
    {
```

```
        delete PageList;
    }

    virtual void LoadPage(string InPage, vector<TableRow*>& PageTableVector,
    int& OutSlotResult) override
    {
        PageAlgorithm::LoadPage(InPage, PageTableVector, OutSlotResult);
        if (PageList->GetListSize() == 0)
        {
            PageList->TableToList(PageTableVector);
        }
        int PageIndex = PageList->FindPageToChange();
        PageTableVector[PageIndex]->VirPageNumber = InPage;
        PageTableVector[PageIndex]->R = 0;

        OutSlotResult = PageIndex;
    };
private:
    RingLinkedList* PageList;
};

class LRUAlgorithm: public PageAlgorithm
{
public:
    virtual void LoadPage(string InPage, vector<TableRow*>& PageTableVector,
    int& OutSlotResult) override
    {
        PageAlgorithm::LoadPage(InPage, PageTableVector, OutSlotResult);
        TableRow* PageToDelete = PagesList.back();

        OutSlotResult = -1;
        for (int i=0; i<PageTableVector.size();i++)
        {
            if (PageTableVector[i] == PageToDelete)
            {
                OutSlotResult = i;
                break;
            }
        }
        #ifdef DEBUG
            if (OutSlotResult == -1)
```

```
            {
                cout << "ERROR Incorrect page to delete" <<
                PageToDelete->VirPageNumber << endl;
            }
        #endif
        PageTableVector[OutSlotResult]->VirPageNumber = InPage;
        PageTableVector[OutSlotResult]->R = 0;

    };
    virtual void PageHasBeenUsed(TableRow* InTableRow) override
    {
        RefreshQueue(InTableRow);
    };
private:
    list<TableRow*> PagesList;

    void RefreshQueue(TableRow* InTableRow)
    {
        auto FindedPage = std::find(PagesList.begin(), PagesList.end(), InTableRow);

        if (FindedPage != PagesList.end())
        {
            PagesList.remove(InTableRow);
            PagesList.push_front(InTableRow);
        }
        else
        {
            if (PagesList.size() == PageBlock)
            {
                PagesList.pop_back();
            }

            PagesList.push_front(InTableRow);
        }
    };
};

// Class for page table
class PageTable
{
public:
```

```
PageTable(PageAlgorithm* InAlgorithm) : Algorithm(InAlgorithm)
{
    for (int i=0; i<PageBlock; i++)
    {
        PageTableVector.push_back(new TableRow);
    }
};
~PageTable()
{
    for (TableRow* Row: PageTableVector)
    {
        delete Row;
    }
    PageTableVector.clear();
};

// Execute command for table
void SendCommandToPage(string InCommand, string InPage)
{
    #ifdef DEBUG
      cout << "Command " << InCommand << " to page " << InPage << endl;
    #endif

    int FindSlotResult = -1;
    int SlotResult = -1;
    if (FindPageSlot(InPage, FindSlotResult))
    {
        #ifdef DEBUG
            cout << "Page number " << InPage << " found in table" << endl;
        #endif

        UsePageSlot(InCommand, FindSlotResult);
    }
    else if (FindEmptySlot(SlotResult))
    {
        #ifdef DEBUG
            cout << "Empty slot has been found at " << SlotResult << endl;
        #endif

        PutNewPageToTable(InPage, SlotResult);
        UsePageSlot(InCommand, SlotResult);
```

```
    }
    else
    {
        #ifdef DEBUG
            cout << "PAGE FAULT: No more empty slots & Page number not
            found in table" << endl;
        #endif

        int OutPageResult;
        Algorithm->LoadPage(InPage, PageTableVector, OutPageResult);
        UsePageSlot(InCommand, OutPageResult);
    }
}

// Clear all R flags for current pages
void ClearRFlags()
{
    for (TableRow* Row: PageTableVector)
    {
        Row->R = 0;
    }
}

// Cout info about every page
void PrintTableStatus()
{
    for (int i=0; i<PageTableVector.size();i++)
    {
        cout << PageTableVector[i]->VirPageNumber;
        #ifdef DEBUG
            cout << " R=" << PageTableVector[i]->R;
        #endif

        if (i != PageTableVector.size()-1)
        {
            cout << " ";
        }
    }
    cout << endl;
};
```

```
private:
    PageAlgorithm* Algorithm;

    vector<TableRow*> PageTableVector;

    // Find # slot in table
    bool FindEmptySlot(int& SlotResult)
    {
        bool IsEmpty = false;
        SlotResult = -1;
        for (int i=0; i<PageTableVector.size();i++)
        {
            if (PageTableVector[i]->VirPageNumber == "#")
            {
                IsEmpty = true;
                SlotResult = i;
                break;
            }
        }
        return IsEmpty;
    };

    // Find InPageNumber in table
    bool FindPageSlot(string InPageNumber, int& SlotResult)
    {
        bool IsFinded = false;
        SlotResult = -1;
        for (int i=0; i<PageTableVector.size();i++)
        {
            if (PageTableVector[i]->VirPageNumber == InPageNumber)
            {
                IsFinded = true;
                SlotResult = i;
                break;
            }
        }
        return IsFinded;
    };

    // Change page number to InPageNumber on InRowIndex
    void PutNewPageToTable(string InPageNumber, int InRowIndex)
```

```
    {
        PageTableVector[InRowIndex]->VirPageNumber = InPageNumber;
    };

    // Execute read/write command for page
    void UsePageSlot(string InCommand, int InRowIndex)
    {
        PageTableVector[InRowIndex]->R = 1;
        Algorithm->PageHasBeenUsed(PageTableVector[InRowIndex]);
    };

};


int main(int argc, char *argv[])
{
    string algorithm;

    if (argc>1)
    {
        algorithm = argv[1];
    }
    else
    {
        return 1;
    }

    PageAlgorithm* CurAlgorithm;

    if (algorithm == "1")
    {
        CurAlgorithm = new ClockAlgorithm;
        #ifdef DEBUG
            cout << "Using Clock" << endl;
        #endif
    }
    else if (algorithm == "2")
    {
        CurAlgorithm = new LRUAlgorithm;
        #ifdef DEBUG
            cout << "Using LRU" << endl;
```

```
        #endif
    }
    else
    {
        return 1;
    }

    string MemmoryCommand;
    PageTable* CurPageTable = new PageTable(CurAlgorithm);

    int CommandNumber = 0;
    while (getline(cin, MemmoryCommand))
    {
        istringstream MemmoryCommandStream(MemmoryCommand);

        string Command, Page;
        MemmoryCommandStream >> Command >> Page;

        CurPageTable->SendCommandToPage(Command, Page);
        CommandNumber++;
        if (CommandNumber % 5 == 0)
        {
            CurPageTable->ClearRFlags();
        }
        CurPageTable->PrintTableStatus();
    }

    delete CurAlgorithm;
    delete CurPageTable;
    return 0;
}
```

6) Выводы

В процессе выполнения лабораторной работы были получены теоретические знания о принципах организации виртуальной памяти и практические навыки создания фрагмента диспетчера памяти и часть функционала операционной системы, отвечающего за замещение страниц при возникновении ошибок отсутствия страниц на языке C++.